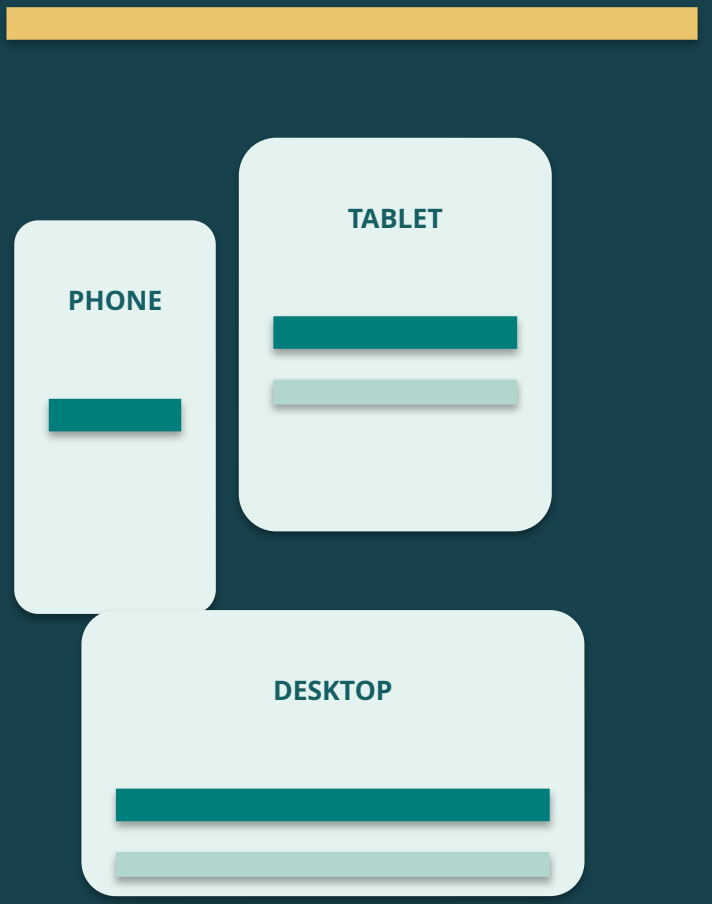
IIA • WEB DESIGN

Page Layout & Responsive Design

Unit 8 • Flexible layouts, the box model, Flexbox and media queries



Library & Information Studies | Records Management Systems

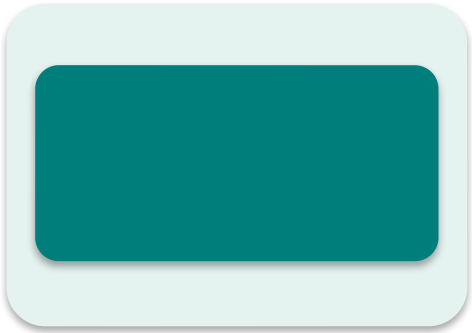
Learning outcomes

1. Explain responsive design and the viewport
2. Describe content, padding, border and margin
3. Use flexible width and box-sizing
4. Arrange cards with Flexbox and gap
5. Apply simple media queries
6. Test usability on different screens

One website, different screen sizes

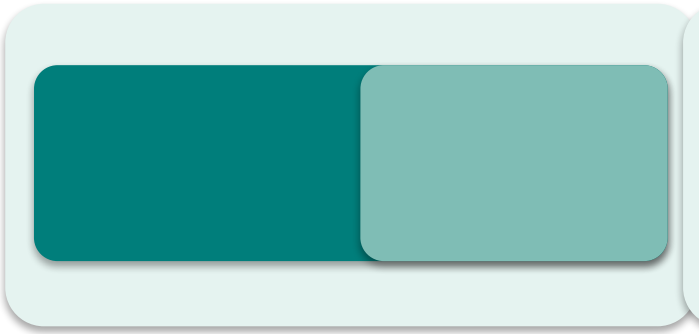
The content stays available; the arrangement changes to suit the screen.

PHONE



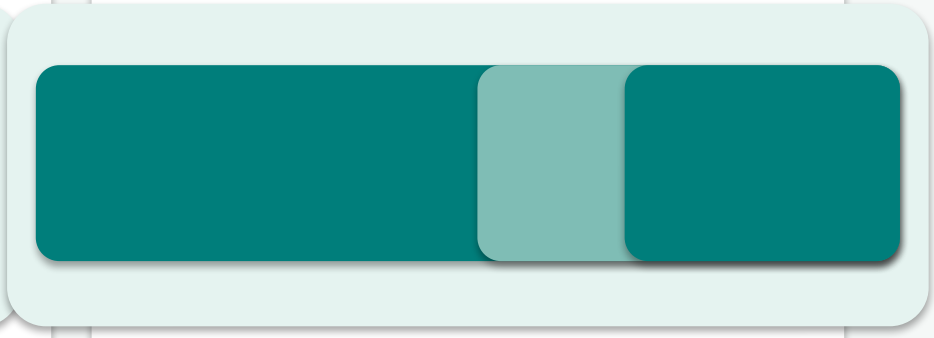
1 column
Readable cards

TABLET



Flexible columns
Wrapped links

DESKTOP



Multiple columns
Comfortable line length

Why layout matters in information services

For library visitors

A student may need opening hours or a catalogue link while walking to a lecture. The webpage should show essential information clearly on a phone.

For records-service visitors

A requester may need to read access procedures on a shared computer or phone. Navigation and instructions should remain usable at different widths.

Fixed versus flexible design

Fixed width

A card with width: 1100px can overflow a 360px viewport. The visitor may have to scroll across the entire page to read one sentence.

Flexible width

A card with width: 100% and a reasonable max-width can use the available space while avoiding excessively long lines on a large monitor.

The viewport: what a browser displays

Viewport

The viewport is the browser area available for the webpage. Two visitors may see the same HTML with very different widths.

Layout response

As available width changes, cards may wrap, menus may stack, and images can scale down. Responsive design does not mean making all text tiny.

The viewport meta tag

Why this matters

The viewport meta tag belongs in the HTML head. It helps mobile browsers size the page for the device.

Keep readable text sizes in CSS; the meta tag does not replace responsive styling.

EXAMPLE

```
<meta name="viewport"
      content="width=device-width,
               initial-scale=1">
```

Mobile-first design

Start small

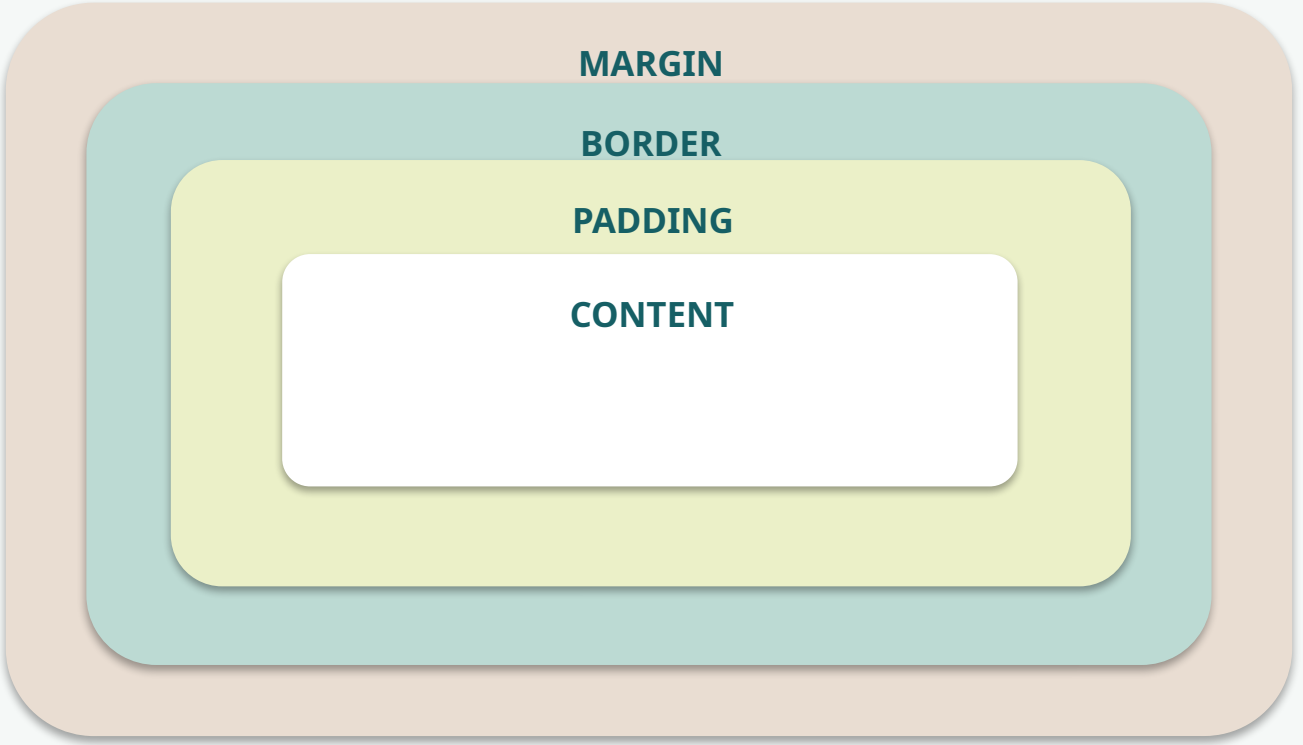
Design the phone-width version first: one clear content column, readable headings and visible navigation links.

Enhance when wider

At a suitable breakpoint, place cards in a row or use more horizontal space. This changes presentation without deleting important content.

The CSS box model

Every standard CSS box has nested layers. Spacing properties refer to different places around its content.



Box model: four layers

Inside the border

Content is text or images. Padding adds room around the content. The border encloses the padded area and can make cards easier to distinguish.

Outside the border

Margin separates one element from neighbouring elements. Its space is outside the border and transparent. Use it for larger separation between sections.

A worked box-model calculation

Default content-box width

320px content + (20px × 2) padding + (2px × 2) border

320 + 40 + 4 = 364 pixels outside the border

Margins add extra space outside this width. With box-sizing: border-box, a declared width of 320px already includes padding and border.

The role of box-sizing

Why this matters

The border-box model usually makes card widths easier to reason about. It does not include margins.

The card’s declared width remains 320px, including its padding and border.

EXAMPLE

```
* { box-sizing: border-box; }

.service-card {
  width: 320px;
  padding: 20px;
  border: 2px solid teal;
}
```

Content width and max-width

Why this matters

A container should use space available on smaller screens but stop growing excessively wide.

max-width is an upper limit; width:100% allows the container to shrink.

EXAMPLE

```
main {
  width: 100%;
  max-width: 1000px;
  margin-inline: auto;
  padding: 20px;
}
```

Spacing with margin and padding

Why this matters

Use padding inside a card and margin between larger sections. This creates an orderly visual hierarchy.

Avoid adding dozens of spaces or blank lines to position elements.

EXAMPLE

```
.service-card {
  padding: 18px;
  border: 1px solid #adc9c5;
}
section { margin-block: 28px; }
```

Why gaps help related items

Gap within a group

Use gap to add consistent space between Flexbox items. It is easier to manage than applying a different margin to each card.

Padding within an item

Use padding so text has breathing room inside a card. A padded card often looks clearer than text pressed against the border.

display: block and inline basics

Block elements

Block-level boxes normally start on a new line and use the available horizontal space. A section or div commonly behaves this way.

Inline elements

Inline boxes normally sit within a line of text. A span or text link can flow alongside surrounding words. Flexbox creates a new layout for its items.

Flexbox: arranging items

Why this matters

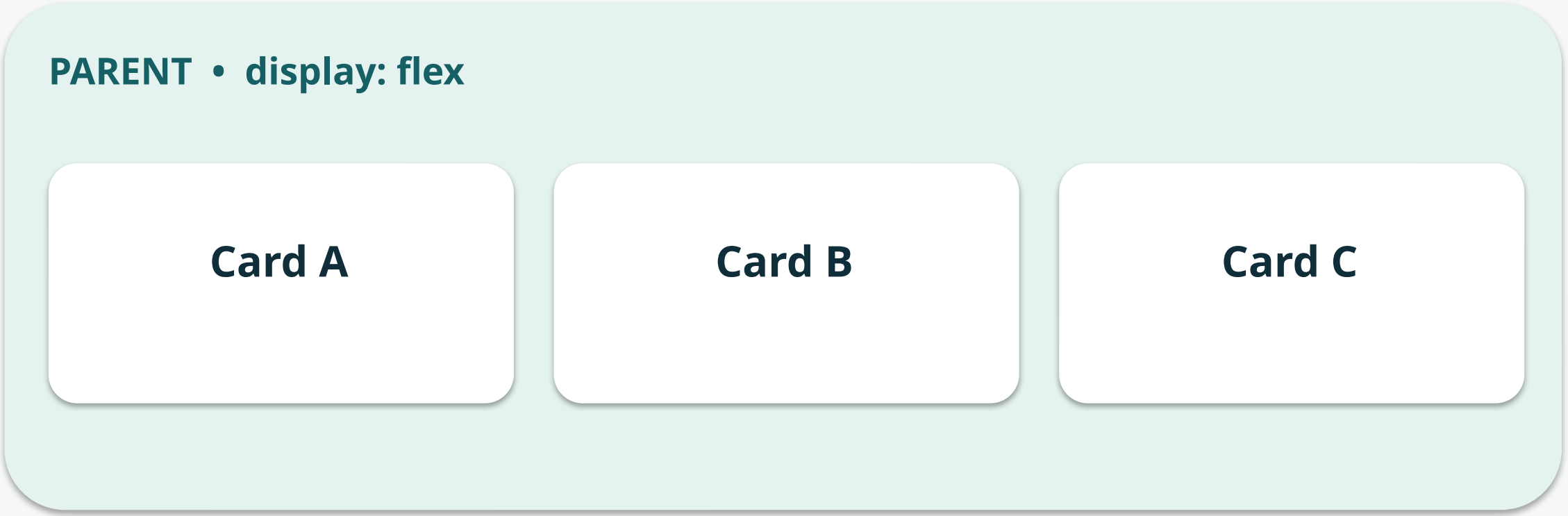
Apply display:flex to the parent container. Its direct children become flex items that the browser can arrange.

Flexbox is useful for menus, service cards and short rows of related content.

EXAMPLE

```
.services {
  display: flex;
}
.service-card {
  /* direct child of .services */
}
```

A Flexbox container and its children



The parent controls the arrangement; each direct child is a flex item.

flex-direction: row and column

Why this matters

row is the common horizontal arrangement. column stacks items vertically. The main axis changes with the flex direction.

A narrow-screen column often makes long service descriptions easier to read.

EXAMPLE

```
.services {
  display: flex;
  flex-direction: column;
}
@media (min-width: 720px) {
 .services { flex-direction: row; }
}
```

gap and flex-wrap

Why this matters

Without wrapping, multiple cards can become cramped. flex-wrap lets them flow onto a new line; gap maintains consistent spacing.

The 240px basis is a preferred size, not a rigid width.

EXAMPLE

```
.services {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}
.service-card { flex: 1 1 240px; }
```

justify-content and align-items

justify-content

Distributes items or free space along the main axis. With flex-direction: row, the main axis is horizontal.

align-items

Aligns items along the cross axis. For a row layout, this usually means vertical alignment. The axes switch with column direction.

Flexible service cards

Why this matters

A flexible basis gives cards room to grow and shrink while avoiding a fixed desktop-only width.

Try the page at phone, tablet and desktop widths; watch the cards wrap.

EXAMPLE

```
.services { display:flex;
  flex-wrap:wrap; gap:16px; }
.service-card {
  flex: 1 1 240px;
  padding: 18px;
}
```

Mobile-first service cards

Why this matters

Make the simplest layout work first. A mobile-first base rule can stack cards, then a media query changes their direction.

The content stays the same; only the layout responds.

EXAMPLE

```
.services {
 display:flex; flex-direction:column;
 gap:16px;
}
@media (min-width:720px) {
 .services {flex-direction:row;}
}
```

Media queries: the basic pattern

Why this matters

Media queries conditionally apply CSS. A min-width condition activates its rules when the viewport is wide enough.

Media queries are part of CSS. They work without JavaScript.

EXAMPLE

```
@media (min-width: 720px) {
  .services {
    flex-direction: row;
  }
}
```

A breakpoint is a design decision

Not one perfect number

Different sites have different content lengths, card sizes and languages. Choose a breakpoint where the layout otherwise becomes cramped.

Test what changes

Resize the browser slowly and observe when text, cards or navigation stop fitting comfortably. Place the breakpoint near that need.

From one column to two

Why this matters

Begin with stacked service cards. At wider widths, allow two cards to sit next to each other. The cards remain in the same HTML order.

Students should be able to read both descriptions in either arrangement.

EXAMPLE

```
.services { display:flex;
  flex-direction:column; gap:16px; }
@media (min-width:720px) {
  .services { flex-direction:row; }
  .service-card { flex:1; }
}
```

Readable navigation on narrow screens

Why this matters

A long horizontal menu may overflow a phone. Wrapping visible links is a simple accessible approach for beginner websites.

Never hide essential links unless an accessible replacement control is provided.

EXAMPLE

```
.site-nav {
  display:flex;
  flex-wrap:wrap;
  gap:12px;
}
.site-nav a { padding:10px 12px; }
```

Responsive images

Why this matters

An image can be visually large on a desktop but should not force a smaller viewport to scroll sideways.

max-width constrains an oversized image; height:auto preserves its proportions.

EXAMPLE

```
img {
  max-width: 100%;
  height: auto;
}
```

Tables on small screens

Why this matters

Wide record inventories may still require horizontal room. Let the table scroll within its wrapper rather than push the whole page sideways.

Preserve table headings and meaningful column structure.

EXAMPLE

```
<div class="table-scroll">
  <table> ... </table>
</div>

.table-scroll {
 overflow-x: auto;
}
```

Avoid horizontal scrolling

Warning signs

A fixed 1100px card on a phone, headings that collide with links, and full-page horizontal scrolling can all prevent a visitor using the site.

Better approaches

Use width:100%, max-width, flexible cards, wrapping menus and a table-specific scrolling container. Test with realistic content.

Professional example: library services

Library services

A library can display catalogue search, borrowing support and opening hours as three cards. On narrow screens, the cards stack for easier reading.

Student task

Open the library example. Resize the browser. Identify when the cards change arrangement and locate the media query in styles.css.

Professional example: records centre

Records services

A records centre may display public enquiries, access procedures and retention guidance. Keep restricted records out of a public demonstration.

Student task

Open the records-centre example. Check whether a visitor can read each service card and use the links on a phone.

Live layout laboratory

Edit and compare

Select a library or records example. Change the gap, padding or breakpoint in the CSS editor, then update the preview.

Three screen widths

Use Phone, Tablet and Desktop controls. Observe whether the cards stack or sit side by side; check text readability and the navigation links.

Guided practical: two service cards

Step 1: open a starter

Choose the library or records-centre example. Open index.html in a browser and locate the service cards.

Step 2: change and test

Edit styles.css, then change the browser width. Compare how the cards and spacing behave without changing the HTML content.

Assignment 8: a small responsive page

Create one page

Make a fictional library or records-centre page with one heading, two service cards and a separate CSS stylesheet.

Show two layouts

Stack the cards on narrow screens and allow two columns on wider screens. Prepare the two files and two screenshots.

Check your design at three widths

PHONE • 360–390px

Is text readable? Are links easy to use? Does the page avoid sideways scrolling?

TABLET • about 768px

Do columns have enough space? Does the breakpoint behave as intended?

DESKTOP • about 1200px

Is line length comfortable? Are cards aligned and separated consistently?

Review: seven key ideas

1. One webpage can fit many screens
2. Use a viewport meta tag
3. Understand the four box layers
4. Prefer flexible widths
5. Flexbox arranges and wraps cards
6. Media queries adapt the layout
7. Test menus, images, tables and text

Ready for usability and evaluation

A good layout is not only attractive.
It is understandable and usable.

Next: Unit 9 — Usability and Website Evaluation